Computational Geometry I

Vectors, cross products, segments, polygons, convex hull

Learning goals

- Represent points/vectors and use **dot** and **cross** products
- Decide **orientation** (left / right / collinear) exactly with integers
- Test **segment intersection**, including all collinear corner cases
- Compute **polygon area** (shoelace) and use **Pick's theorem**
- Test **point in polygon** (with boundary)
- Build the **convex hull** with Andrew's monotone chain in O(n log n)

Why geometry is "hard"

- The math is easy; the **special cases** are not: collinear points, vertical lines, duplicate points, touching segments
- **Precision**: floating point lies — `0.1 + 0.2 != 0.3`
- Golden rule this week: **if the input is integer, stay in integers**
  - cross products of coordinates ≤ 10^9 fit in `long long` (≤ ~$4 \cdot 10^{18}$)
- One tested library of small functions beats re-deriving formulas in a contest

Points and vectors

```
using ll = long long;
struct P {
    ll x, y;
    P operator+(P o) const { return {x + o.x, y + o.y}; }
    P operator-(P o) const { return {x - o.x, y - o.y}; }
    P operator*(ll k) const { return {x * k, y * k}; }
    bool operator<(P o) const { return tie(x, y) < tie(o.x, o.y); }
    bool operator==(P o) const { return x == o.x && y == o.y; }
};
ll dot(P a, P b)   { return a.x * b.x + a.y * b.y; }
ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
ll cross(P o, P a, P b) { return cross(a - o, b - o); }   // turn o→a→b
```

A point and a vector share one type: a point is a vector from the origin.

Dot product

$a \cdot b = a_x b_x + a_y b_y = |a|\,|b| \cos \theta$

sign	meaning
> 0	angle < 90° (same general direction)
= 0	perpendicular
< 0	angle > 90°

Uses: projections, "is the foot of the perpendicular inside the segment?", lengths ($|a|^2 = a \cdot a$).

Cross product

$a \times b = a_x b_y - a_y b_x = |a|\,|b| \sin\theta$ (signed!)

```
      b
     /          a × b > 0 : b is counter-clockwise from a
    /           a × b < 0 : b is clockwise from a
   o-----a      a × b = 0 : collinear (parallel)
```

- $|a \times b|$ = **area of the parallelogram** spanned by a and b
- The sign is the single most important primitive in 2D geometry

Orientation test

`cross(o, a, b) = (a − o) × (b − o)`

- **> 0** — o → a → b turns **left** (counter-clockwise)
- **< 0** — turns **right** (clockwise)
- **= 0** — collinear

CSES 2189 Point Location Test: is p_3 LEFT, RIGHT, or TOUCH of the directed line $p_1 \to p_2$?

```
ll c = cross(p1, p2, p3);
puts(c > 0 ? "LEFT" : c < 0 ? "RIGHT" : "TOUCH");
```

Exact with integers — no epsilon needed.

Point on segment

p lies on segment [a, b] iff

1. collinear: `cross(a, b, p) == 0`, and
2. inside the bounding box: `min(a.x,b.x) ≤ p.x ≤ max(a.x,b.x)` and same for y

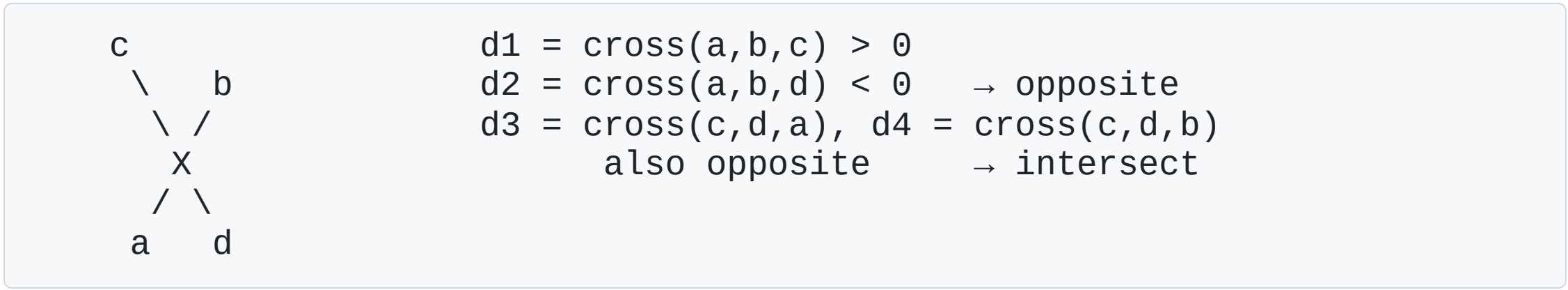
```
bool onSegment(P a, P b, P p) {
    return cross(a, b, p) == 0 &&
           min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
           min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}
```

(Equivalent to condition 2: `dot(a − p, b − p) <= 0`.)

Segment intersection: idea

Segments [a, b] and [c, d] **properly** cross iff

- c and d are on **strictly opposite** sides of line ab, **and**
- a and b are on strictly opposite sides of line cd



Otherwise they intersect only if an **endpoint lies on the other segment**.

Segment intersection: code

```
int sgn(ll v) { return (v > 0) - (v < 0); }

bool segIntersect(P a, P b, P c, P d) {
    int d1 = sgn(cross(a, b, c)), d2 = sgn(cross(a, b, d));
    int d3 = sgn(cross(c, d, a)), d4 = sgn(cross(c, d, b));
    if (d1 * d2 < 0 && d3 * d4 < 0) return true;   // proper crossing
    return onSegment(a, b, c) || onSegment(a, b, d) ||
           onSegment(c, d, a) || onSegment(c, d, b);
}
```

- `sgn` before multiplying: d1 · d2 as raw values could overflow
- Handles touching, overlapping collinear, and degenerate (point) segments

Intersection point of two lines

Lines p + t·u and q + s·v (u × v ≠ 0):

```
t = ((q − p) × v) / (u × v)        point = p + u · t
```

```
// doubles: the intersection point is generally not integral
pair<double,double> lineInter(P p, P u, P q, P v) {
    double t = (double)cross(q - p, v) / cross(u, v);
    return {p.x + u.x * t, p.y + u.y * t};
}
```

Check `cross(u, v) == 0` first (parallel / coincident).

Distances

- Point p to **line** ab: |cross(a, b, p)| / |b – a| (parallelogram area ÷ base)
- Point p to **segment** ab:
  - if `dot(p – a, b – a) < 0` → |p – a|
  - else if `dot(p – b, a – b) < 0` → |p – b|
  - else the line distance

Compare **squared** distances when you can — stays integral.

Polygon area: shoelace formula

Vertices $p_0, p_1, \ldots, p_{n-1}$ in order (either direction):

$$2 \cdot \text{Area} = \left| \sum_i p_i \times p_{i+1} \right| \qquad \text{(indices mod } n\text{)}$$

```cpp
ll area2 = 0;                                   // twice the signed area
for (int i = 0; i < n; i++) area2 += cross(p[i], p[(i + 1) % n]);
// area2 > 0: counter-clockwise order; |area2| / 2 is the area
```

Works for **any simple polygon**, convex or not: triangles fanned from the origin, signed areas cancel outside.

Shoelace: worked example

Polygon (1,1), (4,2), (3,5), (1,4):

i	p_i	p_{i+1}	$p_i \times p_{i+1}$
0	(1,1)	(4,2)	$1 \cdot 2 - 1 \cdot 4 = -2$
1	(4,2)	(3,5)	$4 \cdot 5 - 2 \cdot 3 = 14$
2	(3,5)	(1,4)	$3 \cdot 4 - 5 \cdot 1 = 7$
3	(1,4)	(1,1)	$1 \cdot 1 - 4 \cdot 1 = -3$

Sum = 16 → area = 8, counter-clockwise (positive).

Pick's theorem

For a simple polygon with **integer** vertices:

```
A = I + B/2 − 1
```

- B = lattice points on the boundary = Σ gcd(|Δx|, |Δy|) over edges
- I = interior lattice points = (2A − B + 2) / 2

Example above: 2A = 16; edges give gcd(3,1)+gcd(1,3)+gcd(2,1)+gcd(0,3) = 1+1+1+3 = 6
→ I = (16 − 6 + 2)/2 = **6**. (CSES 2193)

Point in polygon (ray casting)

Shoot a ray from p to the right; count edge crossings. **Odd → inside.**

```
// returns 0 outside, 1 inside, 2 boundary
int inPolygon(const vector<P>& poly, P p) {
    int n = poly.size(); bool in = false;
    for (int i = 0; i < n; i++) {
        P a = poly[i], b = poly[(i + 1) % n];
        if (onSegment(a, b, p)) return 2;
        if ((a.y > p.y) != (b.y > p.y)) {          // edge straddles the ray's line
            ll c = cross(a, b, p);
            if ((b.y > a.y) ? c > 0 : c < 0) in = !in;   // crossing right of p
        }
    }
    return in;
}
```

The half-open test `a.y > p.y` avoids double counting at vertices. O(n) per query.

Convex hull

Smallest convex polygon containing all points.



Andrew's **monotone chain**:

1. Sort points by (x, y)
2. Build the **lower** hull left→right, keeping only left turns
3. Build the **upper** hull right→left the same way
4. Concatenate

Monotone chain: walk-through

Points sorted: A(0,0) B(1,1) C(2,0) D(3,1) E(4,0)

Lower hull (pop while the last turn is **not** a left turn):

add	stack before	action	stack after
A	—	push	A
B	A	push	A B
C	A B	A→B→C right turn → pop B	A C
D	A C	A→C→D left	A C D
E	A C D	C→D→E right → pop D; A→C→E collinear → pop C	A E

Monotone chain: code

```
vector<P> convexHull(vector<P> pts) {      // counter-clockwise, no collinear
    sort(pts.begin(), pts.end());
    pts.erase(unique(pts.begin(), pts.end()), pts.end());
    int n = pts.size();
    if (n < 3) return pts;
    vector<P> h(2 * n);
    int k = 0;
    for (int i = 0; i < n; i++) {                          // lower hull
        while (k >= 2 && cross(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    for (int i = n - 2, lo = k + 1; i >= 0; i--) {      // upper hull
        while (k >= lo && cross(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    h.resize(k - 1);                                       // last == first
    return h;
}
```


O(n log n) for the sort, O(n) for the scan (each point pushed/popped once per

Precision

- Integer input → integer predicates (`long long`; `__int128` if coordinates up to 10^{18} products)
- Unavoidable doubles (intersection points, distances, angles):
  - compare with `EPS` (≈ 1e−9 relative to the scale): `fabs(a - b) < EPS`
  - prefer `long double`, avoid `acos` / `atan` near ±1 / large values
- Prefer `atan2(y, x)` over `atan(y / x)`
- Never use `==` on doubles

More in Regional 8 (precision, robust predicates) and Nationals 8.

Complexity summary

Operation	Time
orientation, on-segment, segment intersection	O(1)
polygon area, perimeter, Pick	O(n)
point in polygon (ray casting)	O(n) per query
convex hull (monotone chain)	O(n log n)
point in **convex** polygon (binary search)	O(log n) — Regional 8

Common pitfalls

- `int` overflow in cross products (coordinates up to 10^9 → products 10^{18})
- Multiplying two cross products directly → overflow; compare signs instead
- Forgetting the collinear-overlap case in segment intersection
- Polygon vertices given clockwise → negative area: take `abs`
- Hull: duplicates, all points collinear, n < 3
- `<= 0` vs `< 0` in the hull: decides whether collinear points are kept
- Using EPS comparisons on integer data (unnecessary and sometimes wrong)

Practice problems

- **CSES 2189** Point Location Test · **CSES 2190** Line Segment Intersection
- **CSES 2191** Polygon Area · **CSES 2192** Point in Polygon
- **CSES 2193** Polygon Lattice Points · **CSES 2195** Convex Hull
- **Kattis** polygonarea · segmentintersection · pointinpolygon · convexhull
- **CF 1C** Ancient Berland Circus (doubles + geometry reasoning)
- **CSES 2194** Minimum Euclidean Distance (stretch: closest pair)

Summary

- Everything reduces to **cross product signs**
- Integer input ⇒ exact predicates; keep doubles for constructed points only
- Segment intersection = 4 orientations + on-segment special cases
- Shoelace for area, Pick for lattice points, ray casting for containment
- Monotone chain: sort + two stacks → convex hull in O(n log n)

Next week: network flow — max flow, min cut, bipartite matching.